

Parallel Methods for Mining Top- K Probabilistic Positive–Negative High-Utility Itemsets from Uncertain Databases: A Shared-Memory Design-Space Study

Huy Vo^{1,2}, Nguyen Le^{1,2†}, Thien Nguyen^{2*†}

¹Faculty of Information Technology, Ton Duc Thang University, Ho Chi Minh City, Vietnam.

²Natural Language Processing and Knowledge Discovery Laboratory, Faculty of Information Technology, Ton Duc Thang University, Ho Chi Minh City, Vietnam.

*Corresponding author(s). E-mail(s): nguyenchithien@tdtu.edu.vn;

†These authors contributed equally to this work.

Abstract

Top- K high-utility itemset mining frees the analyst from choosing a minimum utility threshold, mining over *uncertain* databases models the existence probabilities that arise from noisy or aggregated data, and admitting *negative* item profits captures losses that co-occur with gains. Combining all three yields the Top- K Probabilistic Positive–Negative High-Utility Itemset (PPNHUI) problem, whose enlarged search space makes parallel execution attractive. Yet the *effectiveness* of parallelism is not obvious: the top- K admission threshold only rises as high-utility itemsets are found, and how quickly it rises depends on the data. This paper presents a shared-memory design-space study. We define a common mining core based on a utility–probability list (UPU-List) with a three-tier (**EP**, **PTWU**, **PUB**) pruning scheme, prove that all three tiers remain valid upper bounds when item profits may be negative, and instantiate the core under five parallelisation strategies spanning two design axes: threshold sharing (global vs. thread-local) and load balancing (static, dynamic, work-stealing). Using a *work-ratio* metric (joins performed relative to the sequential baseline) we separate algorithmic effects from raw hardware parallelism. Experiments on four datasets reveal a clear regime spectrum: on dense data (Chess, Mushroom) cooperative threshold escalation lets the work-stealing strategy do as little as one tenth of the

sequential work and reach up to $27.5\times$ speedup on eight cores (super-linear), whereas on sparse retail data (Liquor, Retail) the same strategy performs more work than the baseline and a producer–consumer strategy is the most robust. The non-sharing strategy is consistently the worst and can run slower than sequential. The work-ratio metric explains the entire spectrum and identifies which parallel strategy to prefer for a given regime.

Keywords: high-utility itemset mining, uncertain databases, top- k mining, negative utility, parallel algorithms, shared-memory parallelism

1 Introduction

High-utility itemset mining (HUIM) generalises frequent itemset mining by weighting items with utilities such as profit, revealing patterns that are valuable rather than merely frequent [1, 2]. Three extensions move HUIM closer to real applications. First, the analyst rarely knows a good minimum-utility threshold; *top- K* mining instead asks directly for the K itemsets of highest utility [3, 4]. Second, data gathered from sensors, imputation, or aggregation is *uncertain*: each item in a transaction carries an existence probability [5]. Third, real profit tables contain *negative* entries—loss leaders, penalties, adverse events—so a faithful model must allow items with negative utility [6, 7].

Combining all three yields the Top- K Probabilistic Positive–Negative High-Utility Itemset (PPNHUI) problem. Negative items prevent the search from being pruned by utility alone and so enlarge it, which motivates parallel execution. However, whether parallelism actually helps is subtle. Top- K algorithms prune using an admission threshold that equals the K -th best utility found *so far*; this threshold starts low and rises only as good itemsets are discovered [4]. In a parallel execution many branches are explored before the threshold has risen, creating redundant work; conversely, if several workers *share* the threshold, an early discovery by one worker can prune the others. Which effect dominates is data-dependent, and negative items, by depressing utilities, slow the threshold’s ascent. A study of parallel PPNHUI mining therefore needs a metric that separates algorithmic pruning from raw hardware speedup.

This paper provides such a study for the *shared-memory* setting, where the exact, instantaneous sharing of a global threshold is possible—something a distributed engine cannot easily offer. Our contributions are:

1. We formalise the Top- K PPNHUI problem over uncertain databases (Section 3) and a common mining core built on a utility–probability list (UPU-List) with a three-tier (*EP*, *PTWU*, *PUB*) pruning scheme (Section 4).
2. We prove that all three pruning tiers remain valid upper bounds when item profits may be negative and that the resulting search is exact (Lemmas 1–3, Theorem 4).
3. We instantiate the core under five shared-memory parallel strategies that span two design axes—threshold sharing and load balancing—and analyse them with a *work-ratio* metric.

4. On four datasets we show a regime spectrum: cooperative threshold escalation yields up to $27.5\times$ (super-linear) speedup on dense data but degrades on sparse data, where a producer-consumer strategy is most robust; the non-sharing strategy is always worst and can be slower than sequential (Sections 6–7).

2 Related Work

Top-K HUIM.

Top- K HUIM removes the minimum-utility threshold. Two-phase methods such as TKU [3] and REPT [8] first generate candidates and then verify them, while one-phase methods such as TKO [4] and KHMC [9] use a vertical utility-list [1] and raise the threshold during a single traversal. All of these target *certain* databases with non-negative utilities.

HUIM over uncertain databases.

Lin et al. [5] introduced potential high-utility itemset mining (PHUIM) for uncertain data, with the level-wise PHUI-UP and the list-based PHUI-List built on a probability-utility (PU) list; MUHUI [10] adds pruning to the same structure. These methods mine *all* potential high-utility itemsets rather than a top- K set, and assume non-negative profits.

HUIM with negative utilities.

HUINIV-Mine [6] first handled negative item values; FHN [7] introduced a positive-and-negative utility list (PNU-list) that is far faster, and surveys document the area [11]. Most recently, TopHUI [12] performs *top-K* HUIM with negative utility—but for *certain* databases. Our UPU-List combines the PU-list idea of [5] with the positive/negative bookkeeping of [7]; the structure itself is therefore evolutionary rather than novel, which is why our contribution is the parallel study, not the data structure.

Parallel and distributed HUIM.

Big-data HUIM has largely been distributed: PHUI-Growth on Hadoop [13], PKU for parallel top- K HUIM on Spark [14], P-FHM+ on Spark [15], and GPU-based heuristics [16]. These frameworks partition the work into largely independent subtasks and therefore cannot share an exact global threshold across nodes. Shared-memory multi-core HUIM has received far less attention, and to our knowledge no prior work studies how the choice of shared-memory parallelisation strategy interacts with top- K threshold dynamics under *uncertain, positive-negative* utilities. That gap is the subject of this paper.

3 Preliminaries and Problem Statement

3.1 Basic Concepts

Let $I = \{i_1, \dots, i_m\}$ be a set of items. A *profit table* assigns each $i \in I$ a value $pr(i) \in \mathbb{R}$; unlike classical HUIM, $pr(i)$ may be negative.

Table 1 Example uncertain database. Each cell lists (quantity, probability).

TID	a	b	c
T_1	(1, 0.9)	(2, 0.8)	(1, 1.0)
T_2	(2, 0.6)	–	(3, 0.7)
T_3	–	(1, 0.5)	(2, 0.9)
T_4	(1, 0.8)	(1, 1.0)	–

Definition 1 (Uncertain transaction database) $\mathcal{D} = \{T_1, \dots, T_n\}$ where each transaction T_d associates with every item $i \in T_d$ a quantity $q(i, T_d) \in \mathbb{Z}^+$ and an existence probability $p(i, T_d) \in (0, 1]$. Item probabilities are assumed independent.

Definition 2 (Utility) $u(i, T) = pr(i) \cdot q(i, T)$ and, for $X \subseteq T$, $u(X, T) = \sum_{i \in X} pr(i) q(i, T)$, which may be negative.

Definition 3 (Probabilities) $P(X, T) = \prod_{i \in X} p(i, T)$ for $X \subseteq T$ (and 0 otherwise). The *expected utility* and *existential probability* of X are

$$EU(X) = \sum_{T \supseteq X} u(X, T) P(X, T), \quad EP(X) = 1 - \prod_{T \supseteq X} (1 - P(X, T)). \quad (1)$$

Because $u(X, T)$ may be negative, $EU(X)$ may be negative.

Problem 1 (Top- K PPNHUI mining) Given $\mathcal{D}$, a profit table pr (possibly negative), $K \geq 1$ and $minProb \in [0, 1]$, let $\mathcal{V} = \{X \subseteq I : X \neq \emptyset, EP(X) \geq minProb\}$. Return $\mathcal{R} \subseteq \mathcal{V}$ with $|\mathcal{R}| = \min(K, |\mathcal{V}|)$ such that $EU(X) \geq EU(Y)$ for every $X \in \mathcal{R}$ and $Y \in \mathcal{V} \setminus \mathcal{R}$.

Remark 1 (Negative-utility itemsets) Problem 1 places no sign constraint on EU : when fewer than K valid itemsets have non-negative expected utility, the least-negative ones are returned. This matches the positive–negative setting and our implementation. If an application requires $EU \geq 0$, one adds that constraint to $\mathcal{V}$ and lower-bounds the admission threshold at 0.

3.2 Illustrative Example

Table 1 shows a small uncertain database over items $\{a, b, c\}$ with profits $pr(a) = 5$, $pr(b) = -3$, $pr(c) = 2$ (so b is a negative-profit item); each entry lists an item’s quantity and existence probability. Applying Definition 3, Table 2 gives EU and EP for every itemset. With $minProb = 0.7$ all six itemsets are valid, and ranking them by EU gives $\{a\} > \{a, c\} > \{c\} > \{a, b\} > \{b, c\} > \{b\}$. For $K \leq 4$ the answer contains only positive- EU itemsets, but for $K = 5$ it must include $\{b, c\}$ ($EU = -2.75$) and for $K = 6$ also $\{b\}$ ($EU = -9.3$): a concrete instance of Remark 1. Note also that $EP(\{b\}) = 1$ because b occurs with probability 1 in transaction T_4 .

Table 2 Expected utility and existential probability for every itemset of the database in Table 1. Negative-*EU* itemsets are shown in italics.

Itemset	<i>EU</i>	<i>EP</i>
$\{a\}$	14.50	0.992
$\{a, c\}$	13.02	0.942
$\{c\}$	9.80	1.000
$\{a, b\}$	0.88	0.944
$\{b, c\}$	-2.75	0.890
$\{b\}$	-9.30	1.000

4 Proposed Methods

4.1 Mining Core and Upper Bounds

All five algorithms share one mining core and differ only in how Phase 3 is parallelised. Phase 1 scans $\mathcal{D}$ once to compute per-item bounds and probabilities, discards items that cannot belong to any valid pattern, and builds a UPU-List per surviving item. Phase 2 seeds the result with the valid 1-itemsets. Phase 3 is a depth-first traversal of the set-enumeration tree that extends each prefix by joining UPU-Lists and applies three pruning tiers.

For an itemset X , the UPU-List stores, per transaction $T \supseteq X$, the utility $u(X, T)$, a positive remaining utility $rem(X, T)$, and $\log P(X, T)$ (log-space for stability); two lists are joined by a linear two-pointer merge (Figure 1). Fix a total order $\prec$ on items (descending $PTWU$). With $PTU(T) = \sum_{i \in T, pr(i) > 0} pr(i)q(i, T)$,

$$PTWU(X) = \sum_{T \supseteq X} PTU(T), \quad rem(X, T) = \sum_{i \in T, i \succ X, pr(i) > 0} pr(i)q(i, T), \quad (2)$$

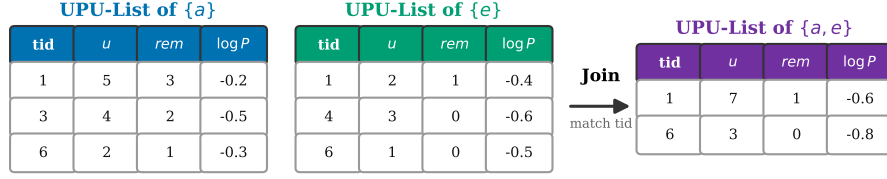
$$PUB(X) = \sum_{T \supseteq X: u(X, T) + rem(X, T) > 0} P(X, T)(u(X, T) + rem(X, T)). \quad (3)$$

An *extension* of X is any $Z \supseteq X$ whose extra items follow X under $\prec$.

Lemma 1 (*EP anti-monotonicity*) *If $X \subseteq Z$ then $EP(Z) \leq EP(X)$.*

Proof $P(Z, T) \leq P(X, T)$ as each $p(i, T) \in (0, 1]$, so $1 - P(Z, T) \geq 1 - P(X, T)$ for all T and the product over T is no smaller for Z ; hence $EP(Z) \leq EP(X)$. $\square$

Lemma 2 (*PTWU bound*) *$EU(Z) \leq PTWU(Z) \leq PTWU(X)$ for any $X \subseteq Z$.*



On matching tid : $u = u_A + u_B$, $rem = \min(rem_A, rem_B)$, $\log P = \log P_A + \log P_B$

Aggregates: $EU = \sum u e^{\log P}$, $PTWU = \min(PTWU_A, PTWU_B)$, $PUB = \sum_{u+rem > 0} e^{\log P(u+rem)}$
 $EP = 1 - \prod (1 - e^{\log P})$ accumulated in log-space

Fig. 1 The UPU-List structure and the two-pointer JOIN. Entries sharing a transaction id are merged, summing utility and log-probability and taking the minimum remaining utility; the join also accumulates the cached aggregates EU , EP , $PTWU$ and PUB .

Proof For $T \supseteq Z$, $u(Z, T)P(Z, T) \leq \max\{u(Z, T), 0\} \leq PTU(T)$: if $u(Z, T) \geq 0$ then multiplying by $P(Z, T) \in (0, 1]$ does not increase it, otherwise the left side is negative; and the positive part of $u(Z, T)$ is at most the positive utilities of T . Summing over $T \supseteq Z$ gives $EU(Z) \leq PTWU(Z)$. Since $\{T \supseteq Z\} \subseteq \{T \supseteq X\}$ and $PTU \geq 0$, $PTWU(Z) \leq PTWU(X)$. $\square$

Lemma 3 (PUB bound) *For every extension Z of X , $EU(Z) \leq PUB(X)$.*

Proof For $T \supseteq Z$ the extra items of Z lie in the $\prec$ -suffix, so their positive contribution is at most $rem(X, T)$ and $u(Z, T) \leq u(X, T) + rem(X, T)$; also $P(Z, T) \leq P(X, T)$ by Lemma 1. Hence $u(Z, T)P(Z, T) \leq \max\{0, u(X, T) + rem(X, T)\}P(X, T)$. Summing over $T \supseteq Z$ and dropping the non-positive terms gives $EU(Z) \leq PUB(X)$. $\square$

Theorem 4 (Correctness) *Pruning X when $EP(X) < minProb$, $PTWU(X) < \tau$, or $PUB(X) < \tau$ —where τ is the current K -th largest admitted EU —never discards an itemset of an exact top- K solution. Hence the search is exact.*

Proof τ only increases and never exceeds the final K -th best EU . By Lemma 1 an itemset failing the EP test and all its extensions are invalid. By Lemmas 2–3, $PTWU(X) < \tau$ or $PUB(X) < \tau$ implies $EU(Z) \leq \tau$ for every extension Z , so none can displace a current top- K member. Thus only subtrees lying entirely below the final threshold are pruned. $\square$

Remark 2 (Why sharing helps and why negatives matter) Theorem 4 holds for any τ not exceeding the final K -th best EU . Raising τ earlier—as when many workers feed a shared

Algorithm 1 GROW: depth-first pattern growth from a prefix

Require: prefix P , start index s , ordered items σ , 1-itemset lists $\mathcal{L}$, collector C , $minProb$

```
1:  $\tau \leftarrow C.THRESHOLD()$ 
2: if  $P.PTWU < \tau$  then return
3: end if
4: for  $j \leftarrow s$  to  $|\sigma| - 1$  do
5:    $J \leftarrow JOIN(P, \mathcal{L}[\sigma[j]], \tau)$ 
6:   if  $J = \text{NULL}$  or  $J.EP < minProb$  or  $J.PTWU < \tau$  or  $J.PUB < \tau$  then
     continue
7:   end if
8:   if  $J.EU \geq \tau$  then  $C.TRYCOLLECT(J)$ 
9:   end if
10:  GROW( $J, j+1, \sigma, \mathcal{L}, C, minProb$ )
11: end for
```

collector—prunes more without ever discarding a solution; a thread-local threshold is simply a smaller, also-safe τ that prunes less. Negative items depress EU and slow the ascent of τ , which both reduces the benefit of sharing and increases redundant parallel exploration.

4.2 Pattern Growth

Algorithm 1 is the shared growth procedure; τ is read from a top- K collector holding the current K highest- EU itemsets.

The growth procedure relies on two further operations: the list JOIN (Algorithm 2), which merges a prefix with an extension item in one linear pass while computing EU , EP , $PTWU$ and PUB ; and TRYCOLLECT (Algorithm 3), the thread-safe admission of a candidate into the shared top- K heap. TRYCOLLECT uses a volatile threshold for a lock-free fast reject and takes the lock only when a candidate might actually be admitted; this is what lets the shared-threshold strategies propagate τ cheaply across workers.

4.3 Five Parallelisation Strategies

The strategies span two axes: whether the admission threshold is *shared*, and how prefix tasks are *balanced* (Figure 2).

- **SEQ** — single thread over all 1-itemset prefixes; the work and time baseline.
- **FJ** — fork-join: prefixes are recursively split into balanced batches on a work-stealing pool; one shared collector, so any worker’s discovery raises τ for all.
- **TPB** — thread-pool: prefixes assigned round-robin by $PTWU$ rank to a fixed pool; shared collector, static one-prefix tasks.
- **PLM** — partition-local merge: $PTWU$ -balanced blocks, each mined into a *thread-local* collector, merged at the end; no threshold sharing.
- **PC** — producer-consumer: a producer enqueues prefix tasks on a bounded queue, consumers mine into a shared collector; dynamic scheduling.

Algorithm 2 JOIN: merge a prefix list with an extension-item list

Require: prefix list A , extension list B (item e), threshold τ

```
1:  $\rho \leftarrow \min(A.PTWU, B.PTWU)$ 
2: if  $\rho < \tau$  then return NULL
3: end if ▷ Tier 2, before the merge
4:  $eu \leftarrow 0; ub \leftarrow 0; lc \leftarrow 0; L \leftarrow \text{empty list}$  ▷  $lc = \sum \log(1 - P)$ 
5:  $i \leftarrow 0; j \leftarrow 0$ 
6: while  $i < |A|$  and  $j < |B|$  do
7:   if  $A.tid_i = B.tid_j$  then
8:      $u \leftarrow A.u_i + B.u_j; r \leftarrow \min(A.r_i, B.r_j); lp \leftarrow A.lp_i + B.lp_j$ 
9:     append  $(A.tid_i, u, r, lp)$  to  $L$ 
10:     $eu \leftarrow eu + u \cdot e^{lp}$ 
11:    if  $u + r > 0$  then  $ub \leftarrow ub + e^{lp}(u + r)$ 
12:    end if
13:     $lc \leftarrow lc + \log(1 - e^{lp})$ 
14:     $i \leftarrow i + 1; j \leftarrow j + 1$ 
15:  else if  $A.tid_i < B.tid_j$  then  $i \leftarrow i + 1$ 
16:  else  $j \leftarrow j + 1$ 
17:  end if
18: end while
19: if  $L$  is empty then return NULL
20: end if
21: return list for  $A \cup \{e\}$  with  $EU=eu, EP=1 - e^{lc}, PTWU=\rho, PUB=ub$ 
```

Algorithm 3 TRYCOLLECT: admit a candidate into the shared top- K

Require: candidate J ; min-heap H ordered by EU ; size K ; volatile threshold τ

```
1: if  $J.EU < \tau$  then return
2: end if ▷ lock-free fast reject
3: acquire lock
4: if  $J.EU \geq \tau$  then
5:    $H.PUSH(J)$ 
6:   if  $|H| > K$  then  $H.POPMIN()$ 
7:   end if
8:    $\tau \leftarrow (|H| < K) ? -\infty : H.MIN().EU$ 
9: end if
10: release lock
```

By Remark 2, FJ, TPB and PC prune against the global K -th best, whereas PLM prunes against weaker local thresholds. SEQ runs the prefixes of Phase 3 in a single loop, and TPB submits one static GROW task per prefix (in $PTWU$ order) to a fixed pool that shares the collector; both are straightforward and omitted. The three non-trivial schedulers are given in Algorithms 4–6. FJ (Algorithm 4) recursively halves the prefix range on a work-stealing pool, balancing load dynamically while sharing one collector. PLM (Algorithm 5) gives each thread a $PTWU$ -balanced block and its *own*

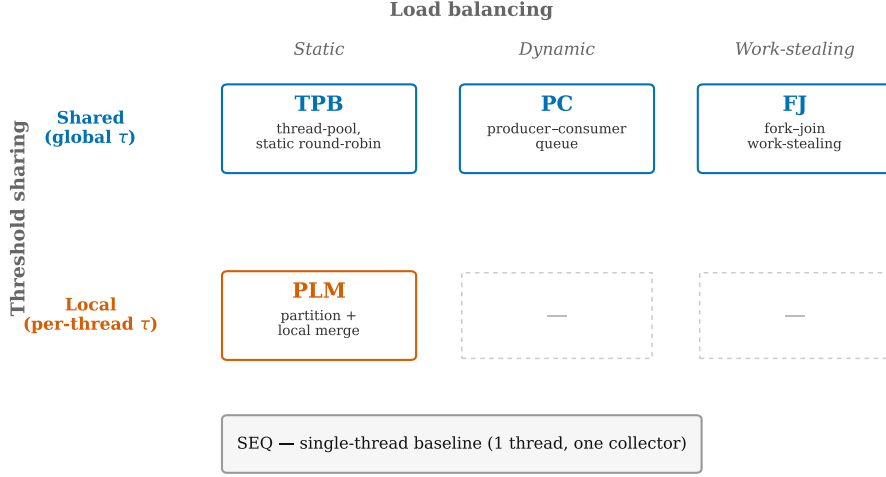


Fig. 2 The shared-memory design space. The five strategies are placed by whether they share the global admission threshold τ (rows) and how they balance prefix tasks (columns). SEQ is the single-thread baseline.

Algorithm 4 FORKJOINMINE (FJ): recursive prefix-batch scheduling

Require: prefix range $[lo, hi]$, ordered items σ , lists $\mathcal{L}$, *shared* collector C , *minProb*

```

1: if  $hi - lo \leq 1$  then
2:   for  $p \leftarrow lo$  to  $hi - 1$  do
3:     GROW( $\mathcal{L}[\sigma[p]]$ ,  $p+1$ ,  $\sigma$ ,  $\mathcal{L}$ ,  $C$ , minProb)
4:   end for
5:   return
6: end if
7:  $m \leftarrow \lfloor (lo + hi)/2 \rfloor$ 
8: fork FORKJOINMINE( $[lo, m), \dots$ ) ▷ stolen by an idle worker
9: FORKJOINMINE( $[m, hi), \dots$ ) ▷ run on the current worker
10: join the forked task

```

collector, merging them only at the end. PC (Algorithm 6) feeds prefixes through a bounded queue to consumer threads that share the collector.

5 Experimental Setup

All algorithms are implemented in Java and share an identical mining core, so performance differences isolate the parallelisation strategy.

Algorithm 5 PARTITIONLOCALMERGE (PLM): no threshold sharing

Require: ordered items σ , lists $\mathcal{L}$, size K , $minProb$, threads B

- 1: partition $\{0, \dots, |\sigma| - 1\}$ into $PTWU$ -balanced blocks $\Pi_1, \dots, \Pi_B$
 - 2: create thread-local collectors $C_1, \dots, C_B$, each of size K
 - 3: **for** $b \leftarrow 1$ **to** B **in parallel do**
 - 4: **for** $p \in \Pi_b$ **do**
 - 5: $GROW(\mathcal{L}[\sigma[p]], p+1, \sigma, \mathcal{L}, C_b, minProb)$
 - 6: **end for**
 - 7: **end for**
 - 8: $C \leftarrow$ empty top- K ; **for** $b = 1$ **to** B : $C.MERGE(C_b)$
 - 9: **return** C
-

Algorithm 6 PRODUCERCONSUMERMINE (PC): dynamic queue scheduling

Require: ordered items σ , lists $\mathcal{L}$, *shared* collector C , $minProb$, threads N

- 1: $Q \leftarrow$ bounded blocking queue
 - 2: **producer:** enqueue $0, \dots, |\sigma| - 1$ into Q , then N poison tokens
 - 3: **for** each of N consumers **in parallel do**
 - 4: **loop**
 - 5: $p \leftarrow Q.TAKE()$
 - 6: **if** p is a poison token **then**
 - 7: **break**
 - 8: **end if**
 - 9: $GROW(\mathcal{L}[\sigma[p]], p+1, \sigma, \mathcal{L}, C, minProb)$
 - 10: **end loop**
 - 11: **end for**
-

Datasets and profit/probability assignment.

We use four transactional benchmarks (Table 3): Chess and Mushroom from the UCI Machine Learning Repository, the Retail dataset of a Belgian retail store, and Liquor from the SPMF repository [17]. They span three orders of magnitude in density. Chess and Mushroom are dense, fixed-length transactions; Retail and Liquor are sparse. To obtain a positive-negative profit table, each item is assigned a profit and a fraction of items is made negative, giving the negative fractions (%neg, 30–51%) in Table 3; profit ranges and average positive/negative profits are summarised in Table 4. For Chess, Mushroom and Retail profits are drawn at random; for Liquor each profit is the item’s real average utility (so positive profits can be large, up to 14,615) and the 30% of items with the lowest utility are negated.

Regarding uncertainty, Chess, Mushroom and Retail are *deterministic*: every existence probability equals 1.0, so they isolate the effects of density and of positive/negative profits at scale (with $minProb = 0.7$ the *EP* filter is inactive on these three). Liquor is the only dataset with *genuine* per-item existence probabilities, taken directly from the SPMF uncertain Liquor data; it is therefore the case that exercises the full probabilistic setting.

Table 3 Characteristics of the experimental datasets. $|T|$: transactions; $|I|$: distinct items; $\text{avg}|t|$, $\text{max}|t|$: average and maximum transaction length; Density = $\text{avg}|t|/|I|$; $|I^-|$: negative-profit items; %neg: their fraction.

Dataset	$ T $	$ I $	$\text{avg} t $	$\text{max} t $	Density	$ I^- $	%neg	Source
Chess	3,196	75	37.00	37	49.33%	38	50.7%	UCI
Mushroom	8,124	119	23.00	23	19.33%	45	37.8%	UCI
Retail	88,162	16,470	10.31	76	0.063%	8,247	50.1%	Belgian retail
Liquor	52,131	4,026	7.87	11	0.196%	1,207	30.0%	SPMF (conv.)

Table 4 Profit-table statistics and the observed EU range of the top- K results (over the swept K).

Dataset	Profit range	Avg. pos.	Avg. neg.	EU range of top- K
Chess	$[-63.0, +48.0]$	+16.22	-17.05	$[43,091, 151,130]$
Mushroom	$[-60.0, +90.0]$	+18.41	-17.38	$[61,761, 451,145]$
Retail	$[-130.0, +130.0]$	+17.26	-17.17	$[3,348, 1,229,567]$
Liquor	$[-27.5, +14,615.0]$	+148.61	-17.55	$[661, 560,291]$

Parameters.

Because the appropriate K differs by density, we sweep $K \in \{10, 20, 30, 40, 50\}$ for Chess, $\{30, \dots, 70\}$ for Mushroom, $\{500, \dots, 900\}$ for Retail, and $\{600, \dots, 1000\}$ for Liquor; $\text{minProb} = 0.7$ throughout. SEQ is the single-thread baseline and the parallel strategies are run at $\{2, 4, 8\}$ threads.

Metrics and protocol.

We report wall-clock time, speedup relative to SEQ, and a *work ratio*: the number of UPU-List joins performed by a strategy divided by that of SEQ at the same $(K, \text{minProb})$. The work ratio separates algorithmic effects (how much of the tree is explored, which threshold sharing changes) from hardware parallelism. Experiments ran on an Intel Core i7-10700K (8 physical cores, 3.8 GHz base) with 32 GB RAM under Java 17; each configuration was executed three times and the median is reported.

6 Results

Across all datasets and configurations every strategy returned exactly K patterns identical to SEQ, consistent with Theorem 4. We focus on performance.

Table 5 reports speedup and work ratio at eight threads for the largest K of each dataset. Figure 3 shows speedup versus K (all K values) at eight threads, and Figure 4 shows the work ratio versus K .

Dense data (Chess, Mushroom).

FJ’s work ratio falls far below one (to 0.11 on Chess at $K = 50$), i.e. cooperative threshold escalation prunes roughly nine tenths of the sequential work. Combined with eight-way hardware parallelism this yields *super-linear* speedup, up to $27.5\times$ on eight

Table 5 Speedup (s) and work ratio (w) at 8 threads, largest K per dataset. Work ratio < 1 means a strategy performs less work than SEQ.

Dataset	K	SEQ (ms)	FJ (s/w)	TPB (s/w)	PC (s/w)	PLM (s/w)
Chess	50	69509	27.5 /0.11	4.7/1.19	4.7/1.23	0.7/7.92
Mushroom	70	7841	10.0 /0.44	8.4/0.52	8.6/0.54	0.9/4.28
Liquor	1000	8499	2.4/1.19	3.1/1.09	3.9 /1.07	2.4/1.66
Retail	900	87442	2.2/1.18	2.7/1.11	3.4 /1.12	0.6/8.95

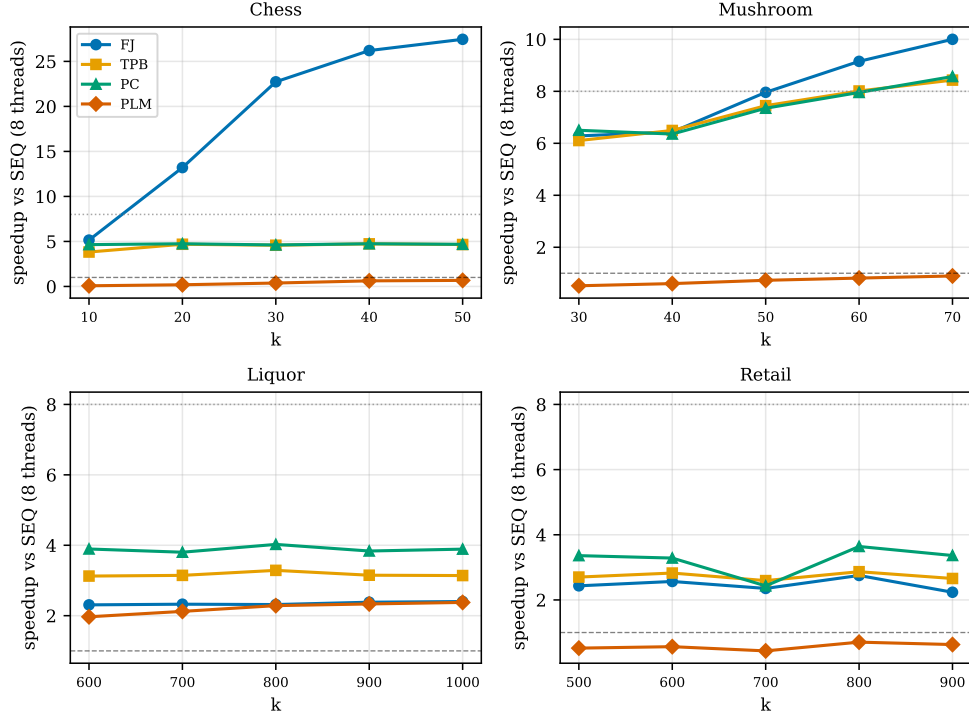


Fig. 3 Speedup over SEQ versus K at eight threads, for all K values per dataset. The dotted line marks eight-way (linear) speedup, the dashed line marks parity with SEQ. FJ is super-linear on the dense datasets and its advantage grows with K , but it is sub-linear on the sparse datasets, where PC scales best.

cores. TPB and PC also scale well ($\approx 4.7\times$ on Chess, $\approx 8.5\times$ on Mushroom) but with higher work ratios than FJ (≈ 1.2 on Chess, ≈ 0.5 on Mushroom), so they capture less of the cooperative-pruning windfall.

Sparse data (Liquor, Retail).

Here FJ’s work ratio exceeds one (1.13–1.32 over the sweep): the threshold rises slowly, so parallel branches explore redundantly before pruning takes effect, and FJ’s speedup

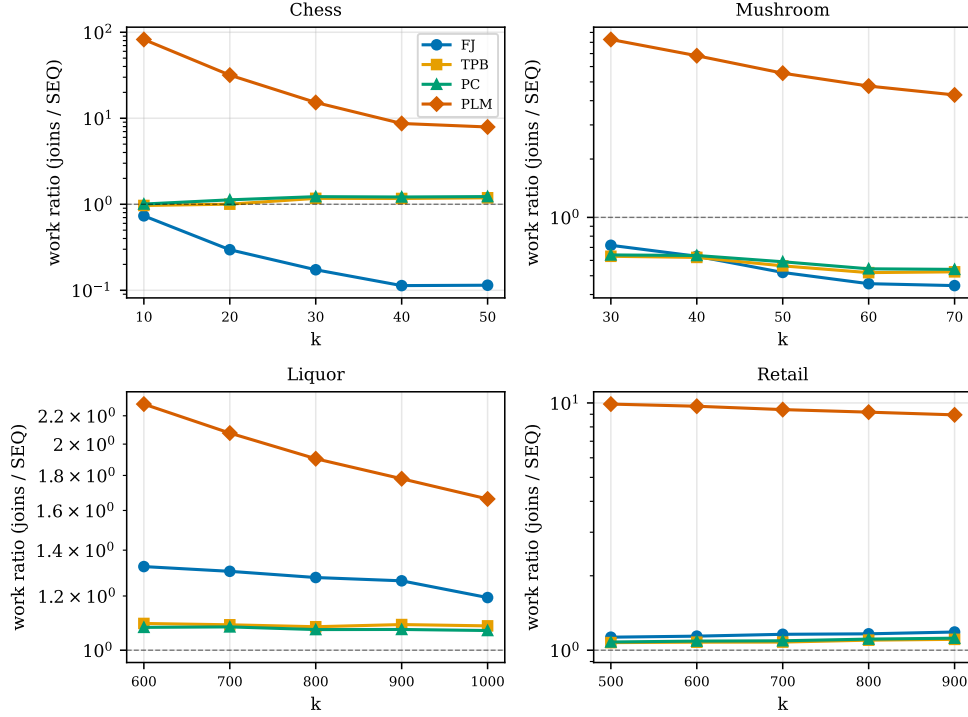


Fig. 4 Work ratio (joins relative to SEQ, log scale) versus K at eight threads. Values below the dashed line ($= 1$) mean less work than sequential. FJ falls well below 1 on dense data and rises above 1 on sparse data; PLM is always highest.

collapses to ≈ 2.2 – $2.8\times$. The producer–consumer strategy PC is the most robust, reaching ≈ 3.4 – $4.0\times$ with a work ratio near one.

The non-sharing strategy.

PLM is the worst strategy in every regime, and on Chess, Mushroom and Retail it runs *slower than sequential*. Its work ratio climbs steeply as K shrinks: on Chess it reaches $\approx 80\times$ the sequential work at $K = 10$ (Figure 4), so PLM can be more than $10\times$ slower than SEQ (speedup as low as $0.07\times$); even at the largest K it still does 4 – $9\times$ the work (speedup 0.6 – $0.9\times$). The penalty tracks how much pruning the shared threshold enables—largest where sharing matters most, and smallest on Liquor, where PLM stays near a $2\times$ speedup.

7 Discussion

The work-ratio metric explains the whole spectrum. The benefit of sharing the admission threshold is proportional to how quickly that threshold rises. On dense data a few high-utility itemsets are found early, τ rises sharply, and a strategy that propagates

τ aggressively (FJ with work-stealing) converts this into a large reduction in explored nodes—enough to make speedup super-linear. On sparse data the utility landscape is flatter, τ rises slowly, sharing buys little, and FJ’s eager parallel splitting instead creates redundant work; the more measured scheduling of PC then wins. PLM, which never shares, forfeits whatever pruning the global threshold would have provided, so it is worst everywhere and worst of all where that pruning is largest.

Two design lessons follow. First, no single strategy dominates: practitioners should pick the strategy from the data regime, and the work ratio—measurable on a small sample—predicts which regime applies. Second, super-linear speedup in top- K mining is an *algorithmic* effect (cooperative pruning), not a hardware one; reporting wall-clock speedup alone is misleading, and the work ratio is needed to attribute it correctly.

On attribution: density, not the negative fraction. The regime tracks structural density almost perfectly (Table 3): the two datasets where FJ wins, Chess and Mushroom, have densities of 49.3% and 19.3%, whereas the two where PC wins, Retail and Liquor, are three orders of magnitude sparser (0.063% and 0.196%). The proportion of negative-profit items does *not* explain the split: Chess (50.7% negative) and Retail (50.1% negative) carry almost the same negative fraction yet fall in opposite regimes, while Liquor has the fewest negatives (30.0%) and still behaves like Retail. Thus, although negative items slow the ascent of τ all else equal (Remark 2), across these datasets it is density—through its effect on how fast a few high-utility itemsets push τ up—that decides which parallel strategy wins, not the negative fraction itself. Disentangling any residual effect of the negative fraction would require holding density fixed while varying it, which we leave to future work.

Threats to validity.

The profits and, for three datasets, the existence probabilities are assigned rather than intrinsic, so they may not reflect a specific application; in particular, only Liquor carries genuine existence probabilities, while Chess, Mushroom and Retail are deterministic ($EP = 1$) and thus exercise the density and positive–negative dimensions but not the probabilistic one. The study is single-machine and shared-memory; and PLM’s partitioning on one dataset (Retail) showed an unusually high work ratio that merits closer inspection.

8 Conclusion

We studied the shared-memory design space for mining top- K probabilistic positive–negative high-utility itemsets. A common UPU-List core with provably sound three-tier pruning was instantiated under five parallel strategies, and a work-ratio metric separated algorithmic pruning from hardware parallelism. Experiments revealed a regime spectrum in which a work-stealing strategy attains super-linear speedup on dense data but is beaten by a producer–consumer strategy on sparse data, while a non-sharing strategy is uniformly worst. The regime is governed by dataset density rather than by the negative-profit fraction. Future work includes an adaptive scheduler that selects a strategy from a cheaply measured work ratio, controlled experiments that

isolate the effect of the negative fraction at fixed density, and a distributed extension that approximates global threshold sharing.

Declarations

Funding. [State funding, e.g. “The authors received no specific funding for this study,” or list grant numbers.]

Competing interests. The authors declare no competing interests.

Ethics approval and consent to participate. Not applicable. This study does not involve human participants, human data, or animals.

Consent for publication. Not applicable.

Data availability. The Chess and Mushroom datasets are available from the UCI Machine Learning Repository, and the Liquor dataset from the SPMF repository [17]; the Retail dataset is the publicly available Belgian retail-store dataset. The profit tables and augmented (positive–negative, and for Liquor uncertain) versions used here are available from the authors on reasonable request. [Add a repository URL/DOI if you deposit them.]

Code availability. The Java implementation of all five strategies and the shared mining core is available from the authors on reasonable request. [Add the public repository URL if applicable.]

Author contributions. H. Vo and N. Le contributed equally: they designed and implemented the algorithms, ran the experiments, and drafted the manuscript. T. Nguyen supervised the work, contributed to the methodology, and revised the manuscript. All authors read and approved the final version.

References

- [1] Liu M, Qu J. Mining high utility itemsets without candidate generation. In: Proc. 21st ACM Int. Conf. on Information and Knowledge Management (CIKM); 2012. p. 55–64.
- [2] Gan W, Lin JCW, Fournier-Viger P, Chao HC, Tseng VS, Yu PS. A survey of utility-oriented pattern mining. IEEE Transactions on Knowledge and Data Engineering. 2021;33(4):1306–1327.
- [3] Wu CW, Shie BE, Tseng VS, Yu PS. Mining top-K high utility itemsets. In: Proc. 18th ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining (KDD); 2012. p. 78–86.
- [4] Tseng VS, Wu CW, Fournier-Viger P, Yu PS. Efficient Algorithms for Mining Top-K High Utility Itemsets. IEEE Transactions on Knowledge and Data Engineering. 2016;28(1):54–67.

- [5] Lin JCW, Gan W, Fournier-Viger P, Hong TP, Tseng VS. Efficient algorithms for mining high-utility itemsets in uncertain databases. *Knowledge-Based Systems*. 2016;96:171–187.
- [6] Chu CJ, Tseng VS, Liang T. An efficient algorithm for mining high utility itemsets with negative item values in large databases. *Applied Mathematics and Computation*. 2009;215(2):767–778.
- [7] Lin JCW, Fournier-Viger P, Gan W. FHN: An efficient algorithm for mining high-utility itemsets with negative unit profits. *Knowledge-Based Systems*. 2016;111:283–298.
- [8] Ryang H, Yun U. Top-k high utility pattern mining with effective threshold raising strategies. *Knowledge-Based Systems*. 2015;76:109–126.
- [9] Duong QH, Liao B, Fournier-Viger P, Dam TL. An efficient algorithm for mining the top-k high utility itemsets, using novel threshold raising and pruning strategies. *Knowledge-Based Systems*. 2016;104:106–122.
- [10] Lin JCW, Gan W, Fournier-Viger P, Hong TP, Tseng VS. Efficiently mining uncertain high-utility itemsets. *Soft Computing*. 2017;21(11):2801–2820.
- [11] Singh K, Singh SS, Kumar A, Biswas B. High utility itemsets mining with negative utility value: A survey. *Journal of Intelligent & Fuzzy Systems*. 2018;35(6):6551–6562.
- [12] Gan W, Wan S, Chen J, Chen CM, Qiu L. TopHUI: Top-k high-utility itemset mining with negative utility. In: *Proc. IEEE Int. Conf. on Big Data (Big Data)*. IEEE; 2020. p. 5350–5359.
- [13] Lin YC, Wu CW, Tseng VS. Mining high utility itemsets in big data. In: *Advances in Knowledge Discovery and Data Mining (PAKDD)*. Springer; 2015. p. 649–661.
- [14] Lin CH, Wu CW, Huang JT, Tseng VS. Parallel Mining of Top-k High Utility Itemsets in Spark In-Memory Computing Architecture. In: *Advances in Knowledge Discovery and Data Mining (PAKDD)*. Springer; 2019. p. 234–246.
- [15] Sethi KK, Ramesh D, Edla DR. P-FHM+: Parallel high utility itemset mining algorithm for big data processing. *Procedia Computer Science*. 2018;132:918–927.
- [16] Fang W, Jiang H, Lu H, Sun J, Wu X, Lin JCW. GPU-based efficient parallel heuristic algorithm for high-utility itemset mining in large transaction datasets. *IEEE Transactions on Knowledge and Data Engineering*. 2024;36(2):652–667.
- [17] Fournier-Viger P, Gomariz A, Gueniche T, Soltani A, Wu CW, Tseng VS. SPMF: a Java open-source pattern mining library. *Journal of Machine Learning Research*. 2014;15:3389–3393.